Accuracy vs. Computational Cost in Nonlinear PDE Solvers

A Comparative Study of Finite Difference, Finite Element, and Spectral Methods on the Viscous Burgers' Equation

Portfolio Project in Numerical Analysis & Scientific Computing

Fluid Dynamics · Python Implementation · Convergence & Performance Analysis

Subject area:	Numerical Methods for PDEs
Language:	Python (NumPy, SciPy, Matplotlib, Streamlit)
Methods:	FDM, FEM, Fourier Pseudospectral
Benchmark PDE:	1D Viscous Burgers' Equation
Duration:	$\approx$ 2 days

This document serves simultaneously as an introduction, a mathematical reference, a methodological guide, and an executable roadmap. It is self-contained: every concept is explained from first principles.

Version 1.0 — March 13, 2026

Contents

1 Introduction and Motivation **3**
1.1 The Big Picture: Why Numerical Methods for PDEs? . . . 3
1.2 Why the Viscous Burgers' Equation? . . . 3
1.3 Why Compare Three Methods? . . . 4
1.4 Project Goals . . . 4

2 Mathematical Foundations **5**
2.1 The Viscous Burgers' Equation: Detailed Analysis . . . 5
2.1.1 The Nonlinear Advection Term: Wave Steepening . . . 5
2.1.2 Conservation Form . . . 5
2.2 The Cole–Hopf Transformation and Exact Solution . . . 5
2.3 Function Spaces and Norms . . . 6

3 Numerical Methods **7**
3.1 Method I: Finite Differences (FDM) . . . 7
3.1.1 Core Idea and Motivation . . . 7
3.1.2 Spatial Discretisation . . . 7
3.1.3 Time Integration: Crank–Nicolson . . . 8
3.1.4 Stability: The CFL Condition . . . 8
3.2 Method II: Finite Elements (FEM) . . . 8
3.2.1 Core Idea and Motivation . . . 8
3.2.2 The Weak (Variational) Formulation . . . 9
3.2.3 Galerkin Discretisation with Linear Elements . . . 9
3.2.4 Matrix Assembly . . . 9
3.2.5 Time Integration . . . 9
3.3 Method III: Fourier Pseudospectral (Spectral) . . . 10
3.3.1 Core Idea and Motivation . . . 10
3.3.2 Fourier Representation . . . 10
3.3.3 Pseudospectral Treatment of Nonlinearity . . . 10
3.3.4 Dealiasing: The 2/3 Rule . . . 11
3.3.5 Time Integration: Integrating Factor Method . . . 11
3.4 Summary: Method Comparison Table . . . 11

4 Analyses to Perform **12**
4.1 Convergence Study . . . 12
4.2 Performance Study . . . 12
4.3 Shock Resolution Study . . . 12
4.4 CFL Sensitivity Study . . . 13

5 Project Roadmap: Step-by-Step Execution Plan **13**

6 Discussion: When to Use Each Method **17**
6.1 Smooth Solutions, Small ν: Spectral Dominates . . . 17
6.2 Shocks and Steep Gradients: FDM with Upwinding . . . 17
6.3 Complex Geometry: FEM Wins . . . 17
6.4 Stiffness and Time-Step Constraints . . . 17

6.5 Practical Decision Matrix 18

7 Deliverables and Portfolio Presentation 18

1 Introduction and Motivation

1.1 The Big Picture: Why Numerical Methods for PDEs?

Many of the most important equations in science and engineering are *partial differential equations* (PDEs). Heat spreads through a metal rod according to the heat equation. Ocean currents and aircraft wings are governed by the incompressible Navier–Stokes equations. Weather prediction depends on the primitive equations of atmospheric dynamics. Financial derivatives are priced through the Black–Scholes PDE. In almost all practical cases, these equations **cannot be solved by hand**: the geometry is too complex, the coefficients vary in space and time, or the nonlinearity makes closed-form solutions impossible.

Numerical methods are the bridge between the mathematical model and a number on a computer screen. They replace the continuous function $u(x,t)$ with a finite set of values at discrete points, and they replace derivatives with algebraic approximations. The art is choosing the right approximation for the right problem.

Why this matters for enterprise / industry

Simulation-driven design, digital twins, and real-time control all rely on efficient PDE solvers. Engineers who understand *why* a solver is accurate (or why it fails) are far more valuable than those who only know how to call a black-box library. This project builds exactly that understanding.

1.2 Why the Viscous Burgers' Equation?

Among all nonlinear PDEs, the **viscous Burgers' equation** occupies a uniquely privileged position as a benchmark problem:

$$\boxed{\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = \nu\,\frac{\partial^2 u}{\partial x^2}, \qquad x \in [0,L],\ t \in [0,T]} \tag{1}$$

where $\nu > 0$ is the *kinematic viscosity*.

Why this equation, and not a simpler one?

1. **Nonlinear advection.** The term $u\,\partial u/\partial x$ is exactly the type of nonlinearity appearing in the Navier–Stokes equations. It causes wave steepening and, in the limit $\nu \to 0$, *shock formation*. This is the single hardest phenomenon for numerical methods to handle correctly.

2. **Diffusion.** The term $\nu\,\partial^2 u/\partial x^2$ is the same parabolic operator appearing in the heat equation. It competes with advection, regularising the solution and preventing infinite gradients when $\nu > 0$.

3. **Exact solution available.** Via the Cole–Hopf transformation (see Section 2.2), the viscous Burgers' equation is exactly solvable. This is rare among nonlinear PDEs and makes it an ideal benchmark: we can measure the *true* numerical error, not just compare solvers against each other.

4. **Tunable difficulty.** By decreasing ν, the solution develops sharper gradients (a *quasi-shock*), stressing every solver differently. This gives us a natural parameter to explore robustness.

5. **Industry recognition.** CFD software validation suites, finite element research papers, and spectral method textbooks universally use Burgers' equation as a first test case. Presenting results on this problem immediately signals familiarity with the field's standards.

Connection to Navier–Stokes

The incompressible Navier–Stokes equations for velocity $\mathbf{u}$ read:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu\,\Delta \mathbf{u}.$$

In 1D with no pressure gradient, this *is* Burgers' equation. Every technique we develop here transfers directly to real CFD.

1.3 Why Compare Three Methods?

No single numerical method dominates all problems. The question an engineer must answer is: *given this PDE, this geometry, and these accuracy requirements, which method gives the best result per unit of computational cost?* This project answers that question concretely by pitting three paradigmatically different approaches against each other:

Method	Core idea	Natural strength
Finite Differences (FDM)	Approximate derivatives by differences	Simple geometry, quick prototypin
Finite Elements (FEM)	Minimise residual over basis functions	Complex geometry, rigorous theor
Spectral (Fourier)	Expand solution in global basis	Smooth solutions, exponential acc

By comparing all three on the *same* problem with the *same* exact solution, we get an honest, reproducible picture of the trade-offs.

1.4 Project Goals

1. Implement three solvers from scratch in Python, with clean, modular code.
2. Derive and implement the exact Cole–Hopf solution for rigorous error measurement.
3. Perform a *convergence study*: measure how the L^2 error decreases as the grid is refined.
4. Perform a *performance study*: measure runtime and memory as a function of problem size.
5. Analyse *shock resolution*: show Gibbs phenomenon, artificial diffusion, and the necessity of upwinding for small ν.
6. Build an interactive **Streamlit visualisation app** to make the results accessible to a non-specialist audience.
7. Produce a clean, reproducible repository suitable for a professional portfolio.

2 Mathematical Foundations

2.1 The Viscous Burgers' Equation: Detailed Analysis

We work on the periodic domain $\Omega = [0, 2\pi]$ with periodic boundary conditions:

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = \nu\,\frac{\partial^2 u}{\partial x^2}, \quad x \in [0, 2\pi],\ t > 0, \tag{2}$$

with initial condition $u(x, 0) = u_0(x)$.

Why periodic? Periodic boundary conditions are the most natural setting for the Fourier spectral method and greatly simplify the analysis. The physics—nonlinear steepening and viscous smoothing—is the same as with other boundary conditions.

2.1.1 The Nonlinear Advection Term: Wave Steepening

Consider the *inviscid* equation ($\nu = 0$), the *Burgers hyperbolic equation*:

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = 0.$$

This is a first-order nonlinear conservation law. Along *characteristics* $\mathrm{d}x/\mathrm{d}t = u$, the solution is constant. Points where u is large travel faster than points where u is small: the wave profile *steepens*. Eventually, characteristics cross, the solution becomes multi-valued, and a **shock** forms in finite time $t^* = -1/\min(u_0')$.

The viscous term $\nu\, u_{xx}$ prevents this: it broadens the shock into a smooth but steep *transition layer* of width $O(\nu)$.

2.1.2 Conservation Form

Equation (1) can be written in *conservation form*:

$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2}{2}\right) = \nu\,\frac{\partial^2 u}{\partial x^2}. \tag{3}$$

This is important: numerical methods that preserve the conservation structure are generally more robust near shocks. The FEM weak form exploits this directly.

2.2 The Cole–Hopf Transformation and Exact Solution

This is one of the most elegant results in nonlinear PDE theory.

Theorem 2.1 (Cole–Hopf, 1950–1951)**.** *Let $\phi(x, t)$ satisfy the* linear *heat equation:*

$$\frac{\partial \phi}{\partial t} = \nu\,\frac{\partial^2 \phi}{\partial x^2}, \tag{4}$$

and define

$$\boxed{u(x, t) = -2\nu\,\frac{\phi_x}{\phi}.} \tag{5}$$

Then u satisfies the viscous Burgers' equation (2).

Derivation sketch. Substitute $u = -2\nu\,\phi_x/\phi$ into (1). After computing u_t, u_x, and u_{xx} using the quotient rule and simplifying, one finds that every term cancels provided ϕ satisfies the heat equation (4). See **(author?)** [9] for the full derivation. □

Why this is powerful. The heat equation is linear and has a known solution via Fourier series. For initial data $\phi_0(x) = \phi(x, 0)$, the heat equation solution on the periodic domain is:

$$\phi(x,t) = \sum_{k=-\infty}^{\infty} \hat{\phi}_k(0)\, e^{-\nu k^2 t}\, e^{ikx}, \tag{6}$$

where $\hat{\phi}_k(0)$ are the Fourier coefficients of ϕ_0.

Computing ϕ_0 from u_0. From (5), $u = -2\nu\,(\ln\phi)_x$, so integrating:

$$\phi_0(x) = \exp\left(-\frac{1}{2\nu}\int_0^x u_0(s)\,\mathrm{d}s\right). \tag{7}$$

Our test case. We use the sinusoidal initial condition:

$$u_0(x) = \sin(x), \quad x \in [0, 2\pi]. \tag{8}$$

This is smooth, periodic, and has a well-understood Cole–Hopf solution, making it the standard benchmark in the literature [1, 4].

The exact solution is:

$$\boxed{u(x,t) = \frac{-2\nu\dfrac{\partial}{\partial x}\ln\phi(x,t)}{1}, \quad \phi(x,t) = \sum_{k=-N_{\text{trunc}}}^{N_{\text{trunc}}} \hat{\phi}_k\, e^{-\nu k^2 t + ikx}} \tag{9}$$

computed numerically with a large truncation $N_{\text{trunc}} = 200$ to represent the infinite series.

Implementation note

In code, ϕ is evaluated on the grid, the derivative ϕ_x is computed via finite differences or FFT, and $u = -2\nu\,\phi_x/\phi$. We must guard against $\phi \approx 0$ (numerical underflow), which is avoided for $\nu > 0$ since $\phi > 0$ always for smooth u_0.

2.3 Function Spaces and Norms

To measure errors rigorously, we need a norm. We use the L^2 **norm**:

$$\|u\|_{L^2} = \left(\int_0^{2\pi} |u(x)|^2\,\mathrm{d}x\right)^{1/2}. \tag{10}$$

Discretely, on a uniform grid $x_j = j\Delta x$ with $j = 0, \ldots, N-1$:

$$\|u\|_{\ell^2} = \sqrt{\Delta x \sum_{j=0}^{N-1} |u_j|^2}. \tag{11}$$

The L^2 **error** at time t is:

$$E(N) = \|u_h(\cdot,t) - u_{\text{exact}}(\cdot,t)\|_{\ell^2}, \tag{12}$$

where u_h is the numerical solution on a mesh of size N (or element count N for FEM).

Remark 2.1 (Why L^2?)**.** The L^2 norm is the natural norm for energy in many physical systems (kinetic energy $\propto \int u^2 \, dx$). It also arises naturally from Galerkin projections (FEM, spectral) and is the standard in convergence analysis. Other choices (L^∞, H^1) can also be informative; we focus on L^2 for simplicity.

3 Numerical Methods

3.1 Method I: Finite Differences (FDM)

3.1.1 Core Idea and Motivation

Finite differences replace derivatives by algebraic combinations of function values at nearby grid points. This is the oldest and most intuitive discretisation technique, dating to Euler. Its main appeal is **simplicity**: given the PDE, one can write a solver in a few dozen lines.

Why FDM first?

FDM is the natural first method to implement: it maps directly to the PDE operators, requires no variational formulation, and its stability analysis (von Neumann, CFL) is the foundation for understanding all other methods.

3.1.2 Spatial Discretisation

Divide $[0, 2\pi]$ into N equally spaced points: $x_j = j\Delta x$, $\Delta x = 2\pi/N$, $j = 0, \ldots, N-1$.

Advection: Upwind scheme. The term $u\,u_x$ is *hyperbolic*: information travels in the direction of u. The **upwind discretisation** respects this:

$$\left(u\frac{\partial u}{\partial x}\right)_j \approx u_j \begin{cases} \dfrac{u_j - u_{j-1}}{\Delta x} & \text{if } u_j > 0 \quad \text{(backward difference)}, \\ \dfrac{u_{j+1} - u_j}{\Delta x} & \text{if } u_j < 0 \quad \text{(forward difference)}. \end{cases} \tag{13}$$

Upwinding is **first-order accurate** in space but crucially *stable*: it adds numerical dissipation in the right direction, preventing spurious oscillations near sharp gradients.

Warning: centred differences for advection are unstable for $\nu \to 0$. The standard second-order centred scheme $(u_{j+1} - u_{j-1})/(2\Delta x)$ can produce growing oscillations when the mesh Péclet number $\mathrm{Pe} = u\Delta x/(2\nu) > 1$. This is a key finding we will observe experimentally.

Diffusion: Second-order centred scheme.

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_j \approx \frac{u_{j+1} - 2u_j + u_{j-1}}{\Delta x^2}. \tag{14}$$

This is second-order accurate and *unconditionally dissipative* (removes energy at all wave numbers), so it is stable in isolation.

3.1.3 Time Integration: Crank–Nicolson

After spatial discretisation, we have a system of ODEs:

$$\frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = \mathsf{A}\mathbf{u} + \mathbf{f}(\mathbf{u}),$$

where A encodes the diffusion operator and $\mathbf{f}$ the nonlinear advection. We use **operator splitting** with:

- **Explicit upwind** for the nonlinear advection (simple, avoids solving a nonlinear system).
- **Crank–Nicolson** (implicit trapezoidal) for diffusion:

$$\frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} = \frac{1}{2}\mathsf{A}(\mathbf{u}^{n+1} + \mathbf{u}^n) + \mathbf{f}(\mathbf{u}^n). \tag{15}$$

  This is *second-order in time* and *unconditionally stable* for the diffusion part.

3.1.4 Stability: The CFL Condition

For the explicit advection step, stability requires the **Courant–Friedrichs–Lewy (CFL) condition**:

$$\mathrm{CFL} = \frac{\max|u|\,\Delta t}{\Delta x} \leq 1. \tag{16}$$

Physically: the numerical domain of dependence must contain the physical domain of dependence. Violating this condition causes exponential blow-up of the solution in finite time.

Convergence expectation for FDM

With upwind advection (1st order) and Crank–Nicolson diffusion (2nd order), the *overall* scheme is **first-order in space**, second-order in time. We expect:

$$E(N) = O(\Delta x) = O(1/N).$$

On a log-log plot of E vs. N, this should appear as a line of slope -1.

3.2 Method II: Finite Elements (FEM)

3.2.1 Core Idea and Motivation

The finite element method reformulates the PDE as a *variational problem*: find u in some function space V such that a weighted integral condition holds for all test functions in V. Basis functions with *local support* are used, leading to sparse matrices.

Why FEM?

FEM is the dominant method in structural mechanics, geophysics, and biomedical simulation. Its variational foundation gives rigorous error bounds and makes it easy to handle complex geometries and mixed boundary conditions. Understanding FEM assembly is an essential skill for any computational scientist.

3.2.2 The Weak (Variational) Formulation

Multiply (1) by a test function $v(x)$ and integrate by parts on $[0, 2\pi]$ with periodic BCs:

$$\int_0^{2\pi} \frac{\partial u}{\partial t}\, v \,\mathrm{d}x + \int_0^{2\pi} u \,\frac{\partial u}{\partial x}\, v \,\mathrm{d}x + \nu \int_0^{2\pi} \frac{\partial u}{\partial x}\,\frac{\partial v}{\partial x}\,\mathrm{d}x = 0. \tag{17}$$

The integration by parts on the diffusion term lowers the smoothness requirement on u from two derivatives to one. This is the weak form.

3.2.3 Galerkin Discretisation with Linear Elements

Partition $[0, 2\pi]$ into N elements of width $h = 2\pi/N$. On each element, u is approximated by *hat functions* $\phi_j(x)$ (piecewise linear, $\phi_j(x_i) = \delta_{ij}$):

$$u_h(x,t) = \sum_{j=0}^{N-1} U_j(t)\,\phi_j(x).$$

Substituting into (17) with $v = \phi_i$ gives:

$$\mathsf{M}\,\dot{\mathbf{U}} + \mathbf{C}(\mathbf{U}) + \nu\,\mathsf{K}\,\mathbf{U} = \mathbf{0}, \tag{18}$$

where:

$$M_{ij} = \int_0^{2\pi} \phi_i\,\phi_j \,\mathrm{d}x \quad \text{(mass matrix, tridiagonal)}, \tag{19}$$

$$K_{ij} = \int_0^{2\pi} \phi_i'\,\phi_j' \,\mathrm{d}x \quad \text{(stiffness matrix, tridiagonal)}, \tag{20}$$

$$C_i(\mathbf{U}) = \int_0^{2\pi} u_h\,\frac{\partial u_h}{\partial x}\,\phi_i \,\mathrm{d}x \quad \text{(nonlinear convection vector)}. \tag{21}$$

3.2.4 Matrix Assembly

The matrices M and K are assembled element by element. For linear elements on a uniform mesh, the local element mass matrix is:

$$\mathsf{M}^{(e)} = \frac{h}{6}\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix},$$

and the local stiffness matrix is:

$$\mathsf{K}^{(e)} = \frac{1}{h}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

These are assembled into global sparse matrices using a connectivity map. With periodic BCs, the global matrices are *circulant tridiagonal*.

3.2.5 Time Integration

The ODE system (18) is integrated using the same Crank–Nicolson/explicit splitting as FDM.

Convergence expectation for FEM

Linear Lagrange elements on a uniform mesh are **second-order accurate in space**:

$$E(N) = O(h^2) = O(1/N^2).$$

On a log-log plot, the slope should be -2.

3.3 Method III: Fourier Pseudospectral (Spectral)

3.3.1 Core Idea and Motivation

Spectral methods expand the solution in a *global* basis of smooth functions (here, Fourier modes) chosen to match the problem's geometry and boundary conditions.

Why spectral?

For smooth periodic problems, spectral methods achieve *exponential* ("spectral") convergence: the error decreases *faster than any polynomial* in $1/N$. This makes them the method of choice for turbulence simulation, weather prediction (ECMWF's spectral model), and oceanography.

3.3.2 Fourier Representation

On $[0, 2\pi]$, any periodic function $u(x,t)$ can be expanded:

$$u(x,t) = \sum_{k=-\infty}^{\infty} \hat{u}_k(t)\, e^{ikx}, \quad \hat{u}_k(t) = \frac{1}{2\pi}\int_0^{2\pi} u(x,t)\, e^{-ikx}\, \mathrm{d}x. \tag{22}$$

Crucially, derivatives are exact in spectral space:

$$\widehat{\left(\frac{\partial u}{\partial x}\right)}_k = ik\,\hat{u}_k, \qquad \widehat{\left(\frac{\partial^2 u}{\partial x^2}\right)}_k = -k^2\,\hat{u}_k.$$

3.3.3 Pseudospectral Treatment of Nonlinearity

The nonlinear term $u\,u_x$ is handled in *physical space*:

1. Compute $u_j = \mathrm{IFFT}(\hat{u}_k)$ at grid points.
2. Compute $(u_x)_j = \mathrm{IFFT}(ik\,\hat{u}_k)$ at grid points.
3. Multiply pointwise: $(u\,u_x)_j = u_j \cdot (u_x)_j$.
4. Transform back: $\widehat{(u\,u_x)}_k = \mathrm{FFT}((u\,u_x)_j)$.

This *pseudospectral* approach avoids computing expensive convolution sums in spectral space. The FFT has cost $O(N \log N)$ vs. $O(N^2)$ for direct convolution.

3.3.4 Dealiasing: The 2/3 Rule

Pointwise multiplication of two functions with $N/2$ modes produces a product with up to N modes. The excess modes *alias* back onto lower modes, creating a spurious energy transfer. The **2/3 rule** removes this:

$$\hat{u}_k \leftarrow 0 \quad \text{for} \quad |k| > \frac{N}{3}. \tag{23}$$

Retaining $N/3$ modes ensures alias-free evaluation of the quadratic nonlinearity.

3.3.5 Time Integration: Integrating Factor Method

The spectral equations are:

$$\frac{\mathrm{d}\hat{u}_k}{\mathrm{d}t} = -\widehat{(u\,u_x)}_k - \nu k^2\,\hat{u}_k.$$

The linear diffusion term $-\nu k^2\,\hat{u}_k$ is *stiff*: for large k, the eigenvalue $-\nu k^2$ can be very negative, forcing tiny time steps for explicit methods. We remove this stiffness via the **integrating factor**:

$$\hat{v}_k(t) = e^{\nu k^2 t}\,\hat{u}_k(t). \tag{24}$$

Then $\hat{v}_k$ satisfies:

$$\frac{\mathrm{d}\hat{v}_k}{\mathrm{d}t} = e^{\nu k^2 t}\left(-\widehat{(u\,u_x)}_k\right),$$

which can be integrated with an **explicit Runge–Kutta** (RK4) scheme without stiffness constraints.

Convergence expectation for Spectral

For smooth periodic initial data, the error decreases exponentially:

$$E(N) = O(e^{-\alpha N}) \quad \text{for some } \alpha > 0.$$

On a *semi-log* plot ($\log E$ vs. N), this appears as a straight line. On a log-log plot, the slope appears to keep increasing—no fixed algebraic rate.

3.4 Summary: Method Comparison Table

Property	FDM	FEM	Spectral
Spatial order	1st (upwind) / 2nd (centred)	2nd (linear)	Exponential
Time order	2nd (CN)	2nd (CN)	4th (RK4)
Matrix structure	Tridiagonal	Tridiagonal sparse	Diagonal
Nonlinearity	Explicit pointwise	Explicit pointwise	Pseudospectral
Shock handling	Good (upwind dissipation)	Moderate	Poor (Gibbs)
Smooth solution	Moderate	Good	Excellent
FFT required	No	No	Yes
Assembly cost	$O(N)$	$O(N)$	$O(N \log N)$ per step

4 Analyses to Perform

4.1 Convergence Study

Goal: Verify the theoretical convergence rates experimentally.
Protocol:

1. Fix $\nu = 0.05$, $T = 1.0$, and a fixed CFL number (adjust Δt proportionally as N increases).
2. Run each solver for $N \in \{16, 32, 64, 128, 256, 512\}$.
3. Compute $E(N) = \|u_h(\cdot, T) - u_{\text{exact}}(\cdot, T)\|_{\ell^2}$.
4. Plot $\log E$ vs. $\log N$. Measure the slope for FDM and FEM. Use semi-log for spectral.

Expected slopes: FDM ≈ -1, FEM ≈ -2, Spectral: decreasing rapidly (not a fixed slope).

4.2 Performance Study

Goal: Measure the computational cost of each method.
Protocol:

1. Use `time.perf_counter()` to measure wall-clock time per time step and total runtime.
2. Use `tracemalloc` to measure peak memory.
3. Vary $N \in \{64, 128, 256, 512, 1024, 2048\}$.
4. Plot runtime and memory vs. N on log-log axes; fit slopes.

Expected scaling:

- FDM: $O(N)$ runtime, $O(N)$ memory.
- FEM: $O(N)$ runtime (sparse solver), $O(N)$ memory.
- Spectral: $O(N \log N)$ runtime (FFT), $O(N)$ memory.

4.3 Shock Resolution Study

Goal: Understand how each method handles near-discontinuities.
Protocol:

1. Reduce ν to $\{0.05, 0.01, 0.005, 0.001\}$.
2. For each ν, run all three solvers at $N = 128$.
3. Plot the solution at $T = 1.5$ (when the shock layer is thin).
4. Observe: FDM upwind smearing, FEM oscillations, Gibbs phenomenon in spectral.

4.4 CFL Sensitivity Study

Goal: Demonstrate the CFL condition as a hard stability boundary.

Protocol:

1. Fix $N = 128$, $\nu = 0.05$.
2. Vary CFL $\in \{0.5, 0.8, 0.95, 1.0, 1.1, 1.5\}$.
3. Run FDM solver; record whether solution blows up.
4. Plot maximum $|u|$ over time for each CFL value.

5 Project Roadmap: Step-by-Step Execution Plan

This section is the actionable plan. Each step builds on the previous. Steps are ordered to minimise debugging time and maximise early visual feedback.

Day 1 — Solvers

Step 1 — Repository Setup and Environment (30 min)

1. Create GitHub repository: `burgers-pde-solvers`
2. Create `conda` environment or `venv`: `numpy`, `scipy`, `matplotlib`, `streamlit`, `tqdm`, `numba` (optional)
3. Create directory structure:

```
burgers-pde-solvers/
  solvers/
    __init__.py
    exact.py       # Cole-Hopf exact solution
    fdm.py         # Finite difference solver
    fem.py         # Finite element solver
    spectral.py    # Pseudospectral solver
  analysis/
    convergence.py
    performance.py
    shock.py
  app.py           # Streamlit application
  notebooks/
    main.ipynb     # Self-contained narrative notebook
  README.md
```

4. Write `README.md` with project description and usage.

Step 2 — Exact Solution Module (45 min)

1. Implement `exact.py`:
   - Function `phi0(x, nu)`: computes ϕ_0 from (7) using numerical integration (`scipy.integrate.cumtrapz`).
   - Function `phi(x, t, nu, N_modes=200)`: evaluates the Fourier series (6) for the heat equation solution.
   - Function `u_exact(x, t, nu)`: computes $u = -2\nu\,\phi_x/\phi$ using FFT differentiation of ϕ.
2. **Test**: plot $u_{\text{exact}}(x, t = 0)$ and verify it matches $\sin(x)$. Plot for $t \in \{0, 0.5, 1.0, 2.0\}$ and observe the progressive decay (viscous smoothing dominates).
3. **Why test this first?** All error measurements depend on the exact solution. A bug here invalidates every subsequent result.

Step 3 — FDM Solver (1 hour)

1. Implement `fdm.py`:
   - Function `build_diffusion_matrix(N, dx, nu)`: assembles the tridiagonal Crank–Nicolson matrix $\mathsf{I} - \frac{\Delta t}{2}\nu\mathsf{D}_2$ (using `scipy.sparse.diags` with periodic wrap-around).
   - Function `upwind_advection(u, dx)`: vectorised upwind advection using `np.where`.
   - Function `solve_fdm(u0, N, T, nu, cfl)`: main time-stepping loop; returns solution array and timing information.
2. **Test**: run with $N = 64$, $\nu = 0.1$, $T = 1$; plot side by side with exact solution.
3. **Validate CFL**: run with CFL $= 1.5$ and confirm blow-up.

Step 4 — FEM Solver (1.5 hours)

1. Implement `fem.py`:
   - Function `assemble_mass(N, h)`: assemble global mass matrix M from local element matrices; use `scipy.sparse.lil_matrix` then convert to CSR.
   - Function `assemble_stiffness(N, h)`: assemble K.
   - Function `convection_vector(U, N, h)`: compute the nonlinear vector $\mathbf{C}(\mathbf{U})$ by element-wise quadrature.
   - Function `solve_fem(u0, N, T, nu, cfl)`: time-stepping loop with Crank–Nicolson for diffusion, explicit for convection; solve $\mathsf{M}\,\delta\mathbf{U} = \mathbf{r}$ using `scipy.sparse.linalg.spsolve`.
2. **Test**: compare with FDM output at $N = 64$; both should agree to within their respective truncation errors.

3. **Key check**: verify mass matrix is symmetric positive definite (`np.linalg.eigvalsh` on the dense version for small N).

Step 5 — Spectral Solver (1 hour)

1. Implement `spectral.py`:
   - Function `wavenumbers(N)`: return array k with correct FFT ordering ($[0, 1, \ldots, N/2-1, -N/2, \ldots, -1]$, using `np.fft.fftfreq`).
   - Function `dealias(u_hat, N)`: zero modes $|k| > N/3$.
   - Function `nonlinear_term(u_hat, k)`: pseudospectral computation of $\widehat{u\,u_x}$.
   - Function `solve_spectral(u0, N, T, nu, cfl)`: RK4 with integrating factor; store solution at output times.
2. **Test**: compare with exact solution at $N = 32$ and $N = 64$; confirm the error is much smaller than FDM/FEM at the same N.
3. **Visualise**: animate all three solvers together using `matplotlib.animation.FuncAnimation`.

Day 2 — Analysis + App

Step 6 — Convergence Analysis (1.5 hours)

1. Implement `analysis/convergence.py`:
   - Loop over methods and grid sizes.
   - For each (N, method): run solver, compute $E(N) = \|u_h - u_{\text{exact}}\|_{\ell^2}$.
   - Store results in a `pandas.DataFrame`; save as CSV.
2. Generate the convergence plot:
   - Log-log plot for FDM and FEM; overlay reference lines of slope -1 and -2.
   - Semi-log plot for spectral; show exponential decay.
   - Measure empirical slopes using `np.polyfit` on $\log E$ vs. $\log N$.
3. **Expected result**: FDM slope ≈ -1, FEM ≈ -2, spectral slope keeps increasing.
4. **Report**: tabulate measured slopes vs. theoretical predictions.

Step 7 — Performance Analysis (1 hour)

1. Implement `analysis/performance.py`:
   - Use `time.perf_counter` to measure total solve time.
   - Use `tracemalloc` for peak memory.
   - Vary $N \in \{64, 128, 256, 512, 1024, 2048\}$.
2. Plot runtime and memory vs. N on log-log axes.

3. Fit power law: `np.polyfit(log N, log t, 1)`; report the exponent.
4. Compute **accuracy-per-second**: $-\log_{10}(E)/\text{runtime}$; which method gives the most accuracy per unit time?

Step 8 — Shock Resolution and CFL Study (1 hour)

1. Implement `analysis/shock.py`:
   - Vary ν from 0.05 down to 0.001; run all three solvers.
   - At each ν, plot the solution at a fixed late time; annotate the shock layer thickness $\delta \approx \nu/|u|$.
   - Highlight Gibbs oscillations in the spectral solution.
2. CFL experiment: run FDM with CFL $\in \{0.5, 0.8, 1.0, 1.1, 1.5\}$; plot $\max|u(t)|$ over time; annotate where blow-up begins.
3. **Key insight to document**: for $\nu = 0.001$, the spectral method produces Gibbs oscillations of $O(1)$ amplitude—the solution looks worse than FDM despite spectral being "more accurate" for smooth solutions. This is a crucial practical lesson.

Step 9 — Streamlit Application (1.5 hours)

1. Implement `app.py` with the following panels:
   - **Sidebar controls**: method selector (checkboxes), ν slider (0.005 to 0.5, log scale), N slider (32 to 512), T slider (0.1 to 3.0), CFL input.
   - **Main panel 1 – Solution Animation**: use `st.pyplot` inside a loop with `time.sleep` to animate, or use `plotly` for a built-in slider.
   - **Main panel 2 – Error Display**: show current L^2 error for each active solver.
   - **Main panel 3 – Convergence Plot**: precomputed from Step 6 data (load CSV); display with Plotly.
   - **Main panel 4 – Performance Table**: precomputed from Step 7 data.
2. **Design goal**: a non-specialist should be able to move the ν slider down to 0.005, see the shock form, and immediately observe Gibbs oscillations in the spectral solution. No reading required.

Step 10 — Jupyter Notebook and Polish (1 hour)

1. Write `notebooks/main.ipynb`: a self-contained narrative that walks through the mathematics, code, and results in order. This is the "paper" version of the project.
2. Each section should have:
   - Markdown explanation (with equations using LaTeX in Markdown cells).
   - Code cells that call the solver modules.

- Output plots inline.

3. Add a `requirements.txt` and a `Makefile` with targets `run-notebook`, `run-app`, `run-tests`.
4. Push to GitHub; add a README with badges, a GIF of the animation, and a link to the live Streamlit app (deploy on Streamlit Community Cloud for free).

6 Discussion: When to Use Each Method

This section provides the *methodological reasoning* that transforms results into understanding—the key differentiator between a software project and a numerical investigation.

6.1 Smooth Solutions, Small ν: Spectral Dominates

When the solution is smooth and periodic, the Fourier coefficients decay rapidly (exponentially for analytic functions). This is precisely when the spectral method earns its exponential convergence rate. For turbulence DNS (Direct Numerical Simulation), spectral methods are the gold standard [3, 2].

6.2 Shocks and Steep Gradients: FDM with Upwinding

Near a shock, the Fourier coefficients decay slowly (algebraically, as $1/k$ for a jump). The spectral representation requires many modes to resolve the sharp feature, and the finite truncation produces Gibbs oscillations—spurious oscillations of $O(1)$ amplitude near the discontinuity. FDM with upwinding introduces *numerical diffusion* that smears the shock, but avoids oscillations. For shock-dominated flows (compressible aerodynamics, explosion modelling), FDM or finite volume methods with shock-capturing schemes are preferred [5].

6.3 Complex Geometry: FEM Wins

FDM requires structured grids (Cartesian or mapped); extending to complex 3D geometries is painful. FEM naturally handles unstructured meshes: triangles in 2D, tetrahedra in 3D. This makes FEM the dominant choice for structural mechanics, geophysical models on spherical shells, and biomedical applications with patient-specific geometries [10].

6.4 Stiffness and Time-Step Constraints

The diffusion term creates stiffness: the stable explicit time step is $\Delta t \leq \Delta x^2/(2\nu)$. For small ν and fine grids, this is not the binding constraint (the CFL condition $\Delta t \leq \Delta x/\max|u|$ dominates). But for the heat equation or parabolic problems with large ν, implicit methods (Crank–Nicolson, BDF2) become essential. The spectral integrating factor handles this elegantly for the diffusion part.

6.5 Practical Decision Matrix

Situation	FDM	FEM	Spectral
Smooth, periodic solution	○	○	●●
Shocks / steep gradients	●●	●	×
Complex geometry	×	●●	×
Quick prototyping	●●	○	●
Very high accuracy needed	○	○	●●
Adaptive refinement	○	●●	○

●● = strong, ● = suitable, ○ = possible, × = unsuitable

7 Deliverables and Portfolio Presentation

Minimum deliverables ("Good Portfolio")

- Clean GitHub repo with `README`, `requirements.txt`, animated GIF.
- Three working solvers with unit tests.
- Convergence plots with measured slopes.
- Short notebook (∼20 cells) with narrative.

Full deliverables ("Research-Assistant Level")

- All of the above, plus:
- Performance (runtime + memory) plots with fitted scaling laws.
- Shock resolution study with Gibbs phenomenon demonstration.
- CFL stability demonstration.
- Methodological discussion (the "when to use what" section).
- Live Streamlit app (deployed on Community Cloud).
- This document as a PDF in the repo.

How to present it in an interview. The strongest framing is: *"I implemented three solvers, benchmarked them rigorously against an exact solution, and quantified the accuracy–cost trade-off. The main finding is that spectral methods are* 10× *more accurate per degree of freedom for smooth solutions, but catastrophically fail near shocks, where upwind FDM provides robust, if diffusive, resolution."* This demonstrates decision-making mindset, not just coding ability.

References

References

[1] Basdevant, C., Deville, M., Haldenwang, P., Lacroix, J. M., Ouazzani, J., Peyret, R., Orlandi, P., and Patera, A. T. (1986). Spectral and finite difference solutions of the Burgers equation. *Computers & Fluids*, 14(1):23–41.

[2] Boyd, J. P. (2001). *Chebyshev and Fourier Spectral Methods*, 2nd edn. Dover Publications, New York.

[3] Canuto, C., Hussaini, M. Y., Quarteroni, A., and Zang, T. A. (2006). *Spectral Methods: Fundamentals in Single Domains*. Springer, Berlin.

[4] Drikakis, D. and Rider, W. (2005). *High-Resolution Methods for Incompressible and Low-Speed Flows*. Springer, Berlin.

[5] LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, Cambridge.

[6] LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, Philadelphia.

[7] Quarteroni, A. and Valli, A. (1994). *Numerical Approximation of Partial Differential Equations*. Springer, Berlin.

[8] Trefethen, L. N. (2000). *Spectral Methods in MATLAB*. SIAM, Philadelphia.

[9] Whitham, G. B. (1999). *Linear and Nonlinear Waves*. Wiley-Interscience, New York.

[10] Zienkiewicz, O. C., Taylor, R. L., and Zhu, J. Z. (2013). *The Finite Element Method: Its Basis and Fundamentals*, 7th edn. Butterworth-Heinemann, Oxford.

[11] Cole, J. D. (1951). On a quasi-linear parabolic equation occurring in aerodynamics. *Quarterly of Applied Mathematics*, 9(3):225–236.

[12] Hopf, E. (1950). The partial differential equation $u_t + uu_x = \mu u_{xx}$. *Communications on Pure and Applied Mathematics*, 3(3):201–230.